PYTHON FOUNDATION CHALLENGE

Week 9: Core Track and Elite Track

Core Track: Questions C1 to C6. Elite Track: Questions E1 to E4. Final Question F is for everyone.

Objective

Week 8 closed with revision, new problems and a stress test. Week 9 is the last week of this phase, and it has two tracks.

- **Core Track:** consolidates the tools you already own (stacks, sorting with counted work, recursion, grids, dictionaries) at a higher bar.
- **Elite Track:** for students who completed Week 8 Question 10 (Big O, hash maps, decorators and recursion). It takes the ideas that question's Hard Bonus only hinted at: bottom-up tables, smarter caches and graphs.
- **Final Question F:** one light, creative project that everyone finishes with.

Track	Who it is for	Questions	Focus
Core	Students who have not yet completed Week 8 Question 10	C1 to C6, then F	Consolidation and higher standards on familiar ideas
Elite	Students who completed Week 8 Question 10	E1 to E4, then F	Dynamic programming, caches, graphs, and proving designs

The tracks are not a ranking. Pick the one that gives you the right amount of struggle. Moving between tracks next phase is fine.

A Word Before You Start

- Some questions are revision with a harder bar: the logic is familiar, the standards are higher.
- Some are new: struggling is the point.
- Bonus questions are optional. Do not touch them until the required ones are done.
- If you are stuck for more than 20 minutes on one question, move on, come back later, or write down the concept that is blocking you. **That note is more valuable than the answer.**

Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean it up afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.
- Test your code with the examples provided **and at least two additional cases of your own, including one edge case** (empty input, one element, duplicates, zero, negative numbers). Elite Track: at least three cases of your own.
- Prefer pure functions: return values from functions and keep print() outside them, unless the question says otherwise.
- Elite Track: put a short comment above every function with its time and space complexity and one sentence of reasoning.

Week 9 at a Glance

Theme	Core Track	Elite Track
Stacks and sorting with counted work	C1, C2	Not covered
Recursion	C3	E1 (replaced by tables), E4
Grids and nested lists	C4, C6	Not covered
Dictionaries and state	C5	E2, E3
Dynamic programming and Big O	Not covered	E1, E4
Graphs	Not covered	E3
Final project	F	F

Core progression: a stack → sorting with counted work → recursion → a spiral through a grid → state in a dictionary → a grid game → your own adventure.

Elite progression: counting from the bottom up → a cache that forgets wisely → searching a network → one problem solved three ways, with proof → your own adventure.

Part 1: Core Track

C1 to C3 revisit earlier topics with a higher bar. C4 to C6 are new. Loops are allowed unless a question says otherwise.

C1. Balanced Brackets (Revision: Use a Stack)

Suggested time: 25 to 30 minutes

Problem

Write `is_balanced(text) -> bool` that returns `True` if every opening bracket `(`, `[` or `{` has a matching closing bracket in the correct order. Ignore every other character.

Examples

```
is_balanced("(a + b) * [c - d]")   # True
is_balanced("{[()]}")              # True
is_balanced("([)]")                # False
is_balanced("((")                  # False
is_balanced("}{")                  # False
is_balanced("")                    # True
```

Constraints

- Use a **list as a stack** (append to push, pop to remove). No recursion.
- Use a **dictionary** that maps each closing bracket to its opening bracket.
- Pure function: no printing inside.
- Do not import libraries.
- Test at least these cases: a closing bracket when the stack is empty, openers left over at the end, and mixed bracket types.

Bonus

Write `max_depth(text)` that returns the deepest nesting level of a balanced string: `max_depth("((a)(b))")` gives 2 and `max_depth("")` gives 0. No `max()`.

What this gauges: Can you recognize when the last thing in must be the first thing out? Can you choose a data structure for the job?

C2. Insertion Sort (Revision: Count Your Work)

Suggested time: 25 to 30 minutes

Problem

Implement `insertion_sort(nums)` that sorts the list **in place** and returns a tuple `(comparisons, shifts)`.

Counting rules: each time you compare the item being inserted (the key) with an element to its left, that is one comparison. Each time you move an element one place to the right, that is one shift.

Examples

```
[5, 2, 4, 1]  -> sorted [1, 2, 4, 5], returns (6, 5)
[1, 2, 3, 4]  -> already sorted, returns (3, 0)
[4, 3, 2, 1]  -> reversed, returns (6, 6)
[]            -> returns (0, 0)
```

Constraints

- No `sort()`, `sorted()`, `min()` or `max()`.
- Sort in place. The function returns only the two counts.
- From **outside** the function, show with the counts that an already sorted list of 10 numbers costs less than a reversed list of 10, then do the same for 20 numbers. Write one sentence on the pattern.
- Do not import libraries.

Bonus

Run last week's bubble sort and this function on the same five lists. Which one does less work on nearly sorted lists? Write one sentence explaining why.

What this gauges: Can you shift items to open a gap and insert into it? Can you measure your own code and see that input order changes the cost?

C3. The Recursion Toolkit (Revision: Base Case First)

Suggested time: 25 to 30 minutes

Problem

Write four recursive functions:

- `digit_sum(n)`: the sum of the digits of a non-negative integer.
- `reverse_text(text)`: the text reversed.
- `count_char(text, ch)`: how many times `ch` appears in `text`.
- `power(base, exp)`: `base` raised to a non-negative whole number `exp`.

Examples

```
digit_sum(1234)                 # 10
digit_sum(0)                    # 0
reverse_text("abc")             # "cba"
reverse_text("")                # ""
count_char("banana", "a")       # 3
power(2, 10)                    # 1024
power(5, 0)                     # 1
```

Constraints

- All four must be **recursive**. No loops inside them.
- No `[::-1]`, `.reverse()`, `.count()`, `pow()` or the `**` operator.
- Above each function, write a comment naming its **base case** and saying how the problem gets smaller each call.
- Pure functions. Do not import libraries.

Bonus

Write `power_fast(base, exp)` that uses the fact that when `exp` is even, `base^exp = (base^(exp/2))^2`. In a comment, explain roughly how many calls it makes for `exp = 1000` compared with `power`.

What this gauges: Do you find the base case first? Do you trust the recursive call to solve the smaller problem?

Part 1B: New Questions

New problems built around grids, dictionaries and state.

C4. Spiral Order

Suggested time: 30 to 35 minutes

Problem

Write `spiral_order(matrix)` that returns all the numbers of a matrix (a list of lists) in spiral order: across the top row, down the right column, back along the bottom row, up the left column, then inward.

Examples

```
spiral_order([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])          # [1, 2, 3, 6, 9, 8, 7, 4, 5]

spiral_order([[1, 2, 3, 4],
              [5, 6, 7, 8]])       # [1, 2, 3, 4, 8, 7, 6, 5]

spiral_order([[1], [2], [3]])      # [1, 2, 3]
spiral_order([])                   # []
```

Constraints

- Pure function. Do not modify the matrix.
- Must work on non-square matrices, a single row, a single column, and an empty matrix.
- Track four boundaries (top, bottom, left, right) and shrink them as you go. No recursion needed.
- No list comprehensions and no libraries.

Hint

After finishing the top row and the right column, check that rows and columns are still left before you walk back along the bottom and up the left. Otherwise you will repeat numbers.

Bonus

Write `spiral_fill(n)` that builds an n by n matrix filled with 1 to n² in a spiral. `spiral_fill(3)` gives `[[1, 2, 3], [8, 9, 4], [7, 6, 5]]`.

What this gauges: Can you manage several moving boundaries at once and handle the awkward shapes at the edges?

C5. Shop Inventory (Dictionaries and State)

Suggested time: 30 to 35 minutes

Problem

A shop's stock is a dictionary of item name to quantity. Write four functions that never change the stock you pass in.

- `restock(stock, item, qty)`: returns a **new** dictionary with `qty` added to the item (adding the item if it is new).
- `sell(stock, item, qty)`: returns a tuple `(new_stock, ok)`. If the item is missing or there is not enough, `ok` is `False` and the stock is unchanged.
- `low_stock(stock, limit)`: returns a list of item names whose quantity is **less than** `limit`, in the dictionary's order.
- `total_items(stock)`: the total number of units in stock.

Quantities must be whole numbers greater than 0, otherwise the operation fails and nothing changes. Selling down to exactly 0 is allowed, and the item stays in the dictionary at 0.

Examples

```
stock = {"rice": 10, "beans": 5}

restock(stock, "gari", 8)    # {"rice": 10, "beans": 5, "gari": 8}
restock(stock, "rice", 5)    # {"rice": 15, "beans": 5}
sell(stock, "rice", 4)       # ({"rice": 6, "beans": 5}, True)
sell(stock, "beans", 9)      # ({"rice": 10, "beans": 5}, False)
sell(stock, "yam", 1)        # ({"rice": 10, "beans": 5}, False)
low_stock(stock, 6)          # ["beans"]
total_items(stock)           # 15
# stock itself is still {"rice": 10, "beans": 5}
```

Constraints

- Build the new dictionary yourself with a loop. No `.copy()`, `.update()`, `dict()` or the `copy` module.
- Prove the original is never changed: print it before and after a successful call and a failed call.
- No `sum()`. Pure functions with printing only outside.
- Test with at least three scenarios of your own, including selling the exact quantity in stock.

Bonus

Add a second dictionary of prices and write `inventory_value(stock, prices)` that returns the total value of stock. Items with no price are skipped.

What this gauges: Can you update state without touching the original? Can you make a failed operation safe?

C6. Tic-Tac-Toe Referee (Grid Logic)

Suggested time: 30 to 35 minutes

Problem

A board is a 3 by 3 list of lists containing `"X"`, `"O"` or `" "` (an empty square). Write `game_status(board)` that returns one of `"X wins"`, `"O wins"`, `"Draw"` or `"In progress"`.

Examples

```
game_status([["X", "X", "X"],
             ["O", "O", " "],
             [" ", " ", " "]])      # "X wins"

game_status([["O", "X", "X"],
             ["X", "O", " "],
             [" ", " ", "O"]])      # "O wins"  (diagonal)

game_status([["X", "O", "X"],
             ["O", "X", "O"],
             ["O", "X", "O"]])      # "Draw"

game_status([["X", " ", " "],
             [" ", "O", " "],
             [" ", " ", " "]])      # "In progress"
```

Constraints

- Check rows and columns with a **loop**, and both diagonals separately. Do not write eight separate if statements for the rows and columns.
- A full board with no winner is a draw. A board with empty squares and no winner is in progress.
- Pure function. Do not modify the board. No libraries.
- Test an empty board and a board where the winning line is in the last row or last column.

Bonus

Return `"Invalid"` if both players have a winning line, or if the number of X and O squares differ by more than one.

What this gauges: Can you walk a grid in several directions with the same loop idea? Can you cover every case without repeating yourself?

Part 2: Elite Track

For students who completed Week 8 Question 10. You have already shown you can combine recursion, hash maps, decorators and Big O. This week there is less hand-holding: some questions only give you the goal and the rules.

- Plan on paper before you code.
- Test with the examples **and at least three cases of your own**, including one edge case.
- Every function is pure: it returns values and prints nothing. Printing lives in the main section only.
- No external libraries (no `functools`, `collections`, `itertools` or `copy`).
- Every function gets a comment above it with its time and space complexity and one sentence of reasoning.

E1. Coins, Bottom-Up

Suggested time: 30 to 35 minutes

Problem

Write two functions that use a **table (a list), not recursion**:

- `count_ways_dp(amount, coins)`: the number of combinations of coins that make `amount` (order does not matter, coins can be reused). This is last week's `count_ways`, rewritten.
- `min_coins(amount, coins)`: the **fewest** coins needed to make `amount`, or `-1` if it is impossible.

Examples

```
count_ways_dp(5, [1, 2, 5])             # 4
count_ways_dp(100, [1, 5, 10, 25, 50])  # 292
count_ways_dp(0, [1, 2])                # 1
min_coins(11, [1, 2, 5])                # 3   (5 + 5 + 1)
min_coins(100, [1, 5, 10, 25, 50])      # 2   (50 + 50)
min_coins(3, [2])                       # -1
min_coins(0, [1])                       # 0
```

Constraints

- Both functions use a list of size `amount + 1` and loops. **No recursion.**
- Do not modify the `coins` list.
- Handle: empty coins, amount 0, and coins larger than the amount.
- Run `count_ways_dp(10000, [1, 5, 10, 25, 50])`. It must finish quickly. In a comment, explain why last week's recursive version could not.

Hint

For `min_coins`, ask: "What is the best way to make a, if my last coin was c?"

Bonus

Make `min_coins` also return **which coins** were used, for example `(3, [5, 5, 1])`.

What this gauges: Can you turn a recursive idea into a table that fills from the bottom up? Can you explain why it is faster and why it never hits the recursion limit?

E2. The Cache That Forgets Wisely

Suggested time: 35 to 40 minutes

Problem

Write a decorator factory `lru_cache(max_size)` that evicts the **least recently used** entry, not the oldest inserted one. Whenever a cached key is **hit**, it becomes the "newest".

Expose `wrapper.hits`, `wrapper.misses`, `wrapper.calls` and a function `wrapper.keys()` that returns the cached keys from **oldest to newest**.

Also write a second decorator `fifo_cache(max_size)` with the same interface that evicts the **oldest inserted** entry (your Week 8 `memoize`).

Example: use a function `f(x)` that returns `x * x` with `max_size = 2` and call it with the sequence `1, 2, 1, 3, 2, 1`.

Examples

```
lru_cache:  hits = 1, misses = 5
fifo_cache: hits = 2, misses = 4
```

Constraints

- The cache is a **dictionary in the closure**. You may not use `OrderedDict`. Python dictionaries remember insertion order, so decide how to "move" a key to the newest position.
- No global variables. No libraries.
- `max_size` of 0 means nothing is ever cached. `max_size=None` means unlimited.
- Must work with both one argument and several arguments.

Written Analysis (as comments)

1. On the example above, FIFO did **better** than LRU. Does that mean FIFO is the better policy? Explain what kind of access pattern favours each.
2. What is the time complexity of a cache hit in your LRU? Where could it become O(n), and how did you avoid that?
3. Describe one real system where forgetting the least recently used item is the right call.

Bonus

Add `wrapper.clear()` that empties the cache and resets the counters. Then add a limit by counting calls (not time): entries older than k calls are removed.

What this gauges: Can you manage state in a closure, change a policy without breaking the interface, and judge a design by its access pattern rather than a single test?

E3. Shortest Route

Suggested time: 35 to 40 minutes

Problem

A network is a dictionary where each key is a node and its value is the list of nodes it connects to. Connections are one-way (a **directed** graph).

```
graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D", "E"],
    "D": ["F"],
    "E": ["F"],
    "F": []
}
```

Write:

- `shortest_path(graph, start, goal)`: returns a list of nodes for one shortest path from `start` to `goal`, or `[]` if there is no path.
- `reachable(graph, start)`: returns the **number of nodes** reachable from `start`, including `start`.
- `has_cycle(graph)`: returns `True` if the graph contains a cycle.

Examples

```
shortest_path(graph, "A", "F")   # ["A", "B", "D", "F"]  (any shortest path is fine)
shortest_path(graph, "A", "A")   # ["A"]
shortest_path(graph, "F", "A")   # []
reachable(graph, "A")            # 6
reachable(graph, "C")            # 4
has_cycle(graph)                 # False
has_cycle({"X": ["Y"], "Y": ["X"]})   # True
```

Constraints

- Use a **dictionary** to remember which nodes you have visited and how you got to them.
- For `shortest_path`, use a **queue implemented with a list and an index pointer**. **Do not use ¦pop(0)¦.** In a comment, explain why `pop(0)` would be slow.
- No `collections.deque`. No libraries.
- Handle: an unknown start or goal node, a node with no connections, and a graph with loops.
- Do not modify the graph.

Hint

Explore the start's neighbours first, then their neighbours, level by level. The first time you reach the goal is the shortest way there.

Bonus

Write `all_shortest_paths(graph, start, goal)` that returns **every** shortest path.

What this gauges: Can you model a problem as a network, use a hash map to avoid repeated work, and choose a data structure based on its cost?

E4. Edit Distance: One Problem, Three Solutions

Suggested time: 45 to 60 minutes

Tests: recursion, memoization, bottom-up tables, Big O and measurement, all in one program.

Problem

The **edit distance** between two strings is the smallest number of single-letter edits (insert, delete or replace one letter) needed to turn one into the other.

```
edit_distance("kitten", "sitting")  # 3
edit_distance("", "abc")            # 3
edit_distance("same", "same")       # 0
edit_distance("flaw", "lawn")       # 2
```

Solve it **three ways**:

1. `edit_naive(a, b)`: plain recursion, no caching.
2. `edit_memo(a, b)`: the same recursion, using a decorator you wrote (Week 8 `memoize` or E2's `lru_cache`).
3. `edit_table(a, b)`: bottom-up with a table (a list of lists).

Constraints

- All three must return identical answers on every test.
- Count the **function calls** made by `edit_naive` and `edit_memo` on `"kitten"` vs `"sitting"` and on two longer pairs of your own (up to 8 letters). Print the counts side by side. Keep the counter in a mutable argument or a function attribute, as in Week 8's `fib`. No global variables.
- Print a table with columns **function | result | calls (or table cells) | time complexity | space complexity**, built with string logic only, for two pairs of strings you create.
- Decorators are allowed for memoizing only. No `functools`.
- Pure functions. Printing only in the main section.
- Do not hardcode answers.

Written Analysis (as comments)

1. What is the time complexity of the naive version? Why does it blow up?
2. What is the time and space complexity of the memoized version? Of the bottom-up version?
3. For two strings of length 5000, which version would you use and why? Think about the recursion limit and memory.
4. Your table has `len(a)+1` rows and `len(b)+1` columns. Which **single previous row** does each step actually need? How could that change your space complexity?

Hard Bonus

Rewrite `edit_table` using only **two rows** of memory, and prove it returns the same answers as the full table on at least five pairs.

What this gauges: Can you solve one problem in several ways and **prove** they agree? Can you measure growth rather than just recite it? Can you explain a trade-off between speed, memory and safety?

Part 3: Final Question (Everyone)

F. Your Own Adventure

Suggested time: 30 to 40 minutes

Tests: nothing new. This one is a victory lap. Use what you already know to build something you would want to play.

A Word Before You Start

This is not a stress test. There are no hidden edge cases, no Big O and no trick constraints. You have spent weeks solving problems other people designed. Now you design one.

Problem

Build a small story game where the player makes choices and the story changes. Write a text adventure with **at least 5 rooms (or scenes)** and **at least 2 different endings**: one good, one bad. Pick any theme you like: a haunted compound, a trip through Lagos traffic, a jungle, a space station, a school where the teacher has vanished. Make it funny, scary or silly.

Example

```
You wake up in a dark room. There is a door to the NORTH and a window to the EAST.
> north
You enter a kitchen. A strange smell is coming from the fridge.
Do you OPEN the fridge or go BACK?
> open
Inside the fridge is a golden key!
```

Requirements

- At least **5 rooms** and **2 endings**: one win and one lose.
- Store your rooms in a **dictionary**, for example `rooms = {"kitchen": {"text": "...", "exits": {"back": "hallway"}}}`.
- Use a **loop** that keeps the game running until the player wins, loses, or types `quit`.
- Track at least **one item** the player can pick up (a key, a map, a torch) in a **list**, and make that item matter somewhere, such as a locked door or a safe path.
- If the player types something invalid, the game should say so kindly and **not crash**.
- No external libraries. No random: the story should be the same every time.

Keep It Clean (Optional but Nice)

If you want to practise what you have learned, keep your game logic in a small function such as `move(current_room, command)` that returns the next room, and keep `print()` and `input()` in your main loop. If your first version has everything mixed together, that is fine. Make it work first.

Bonus (Only If You Are Having Fun)

- Add a **second item** and a puzzle that needs both.
- Add a **health or turn counter**, for example you lose after 10 moves.
- Add a **secret room** that only a curious player will find.
- Let a friend play it and watch where they get stuck.

How This Will Be Judged

Not on difficulty. Does it run without crashing? Is the story fun to read? Is there at least one choice that changes what happens?

> **A note to finish.** You started this phase building a simple calculator. Now you are building a world someone else can play in. Share your game with the person next to you and play theirs. Enjoy it. You have earned it.

Time Guide

Track	In class	Suggested time	Take-home
Core	C1, C3, C5, C6, F	2 hrs 20 min to 2 hrs 50 min	C2, C4
Elite	E1, E3, E4, F	2 hrs 20 min to 2 hrs 55 min	E2, and any written analysis not finished

Track	Time for every question
Core	C1 to C6: 2 hrs 45 min to 3 hrs 15 min. With F: 3 hrs 15 min to 3 hrs 55 min.
Elite	E1 to E4: 2 hrs 25 min to 2 hrs 55 min. With F: 2 hrs 55 min to 3 hrs 35 min.

The session runs 3 hrs 15 min, so the in-class column fits with a little slack. Finish each question inside its suggested time. If you pass the top of the range, write down what blocked you and move on. Take-home questions are due at the next session.

Self-Assessment Guide

Core Track

Question	Concept being tested	What it tells you
C1. Balanced Brackets	Stack, dictionary lookup	Do you know when last in must be first out?
C2. Insertion Sort	Shifting, counting work	Can you measure your own code?
C3. Recursion Toolkit	Base case, shrinking problem	Do you trust the recursive call?
C4. Spiral Order	Boundaries, grid edges	Can you manage several moving limits at once?
C5. Shop Inventory	Dictionaries, no mutation	Can you update state without touching the original?
C6. Tic-Tac-Toe Referee	Grid logic, state	Can you cover every case without repeating yourself?
F. Your Own Adventure	Dictionaries, loops, items	Can you turn what you know into something real?

Elite Track

Question	Concept being tested	What it tells you
E1. Coins, Bottom-Up	Bottom-up tables	Can you build the answer from small cases up?
E2. The Cache That Forgets Wisely	State, policy, closures	Can you change a design without breaking its interface?

Question	Concept being tested	What it tells you
E3. Shortest Route	Graphs, queues, hash maps	Can you model a real-world network?
E4. Edit Distance	Integration, Big O	Can you solve, prove and explain?
F. Your Own Adventure	Dictionaries, loops, items	Can you turn what you know into something real?

How to Read Your Results

Core Track

- **5 to 6 required questions solved cleanly:** Strong. You are testing carefully and handling edge cases.
- **3 to 4 solved:** Solid fundamentals. Keep practising the questions that blocked you.
- **1 to 2 solved:** Useful information, not failure. Note which concept blocked you (stacks, recursion, grids or dictionaries) and bring that list to the session.
- **C3 solved:** You can think recursively and name your base case.
- **F finished and a friend can complete it:** You can build something other people can use.

Elite Track

- **E1 to E3 solved cleanly:** You are comfortable with the core tools of algorithmic thinking.
- **E4 solved with a correct analysis:** You can compare designs and defend your choice. That is the skill that matters most.
- **Hard Bonus solved:** You are thinking about memory as well as speed.

A Note on “Gauging”

This challenge is a gauge, not an exam. The point is to find out which ideas feel natural and which need another pass. Struggling is data, not failure. If you finish half of this and can name exactly which concept blocked you on the rest, you have learned more than someone who copied answers to all of it.

Write down your blockers. Bring them to the next session. That list is the real deliverable.

Notes for the Instructor

Design choice	Reason
Two tracks with a shared final question	Only some students completed Week 8 Question 10, so each group gets the right level of struggle and everyone ends on the same light project.
C1 to C3 revisit stacks, sorting and recursion	Balanced parentheses and insertion sort come from the exercise list and have not been revisited yet. C3 repeats the recursion habit of naming a base case.
C4 to C6 use grids, dictionaries and state	They are new problems that practise the same habits as earlier weeks: pure functions, no mutation, and edge cases.

Design choice	Reason
E1 rewrites last week's count_ways bottom-up	Closes the Week 8 Hard Bonus and shows why tables beat deep recursion.
E2 includes the FIFO vs LRU comparison	The example is built so FIFO scores better, which forces students to think about access patterns instead of assuming LRU always wins.
E3 bans pop(0)	Introduces a data-structure cost lesson (O(n) vs O(1)) with a real consequence.
E4 solves one problem three ways	Mirrors Week 8's Question 10: recursion, caching, measurement and Big O in one program.
Final question F is a text adventure	A light, creative close to the phase. It practises dictionaries, loops and lists without any new concept.

Expected values

Question	Expected results
C1	is_balanced: True, True, False, False, False, True for the six examples in order.
C2	insertion_sort: [5,2,4,1] returns (6, 5); [1,2,3,4] returns (3, 0); [4,3,2,1] returns (6, 6); [] returns (0, 0).
C3	digit_sum(1234) = 10; count_char("banana", "a") = 3; power(2, 10) = 1024.
C4	3 by 3: [1,2,3,6,9,8,7,4,5]; 2 by 4: [1,2,3,4,8,7,6,5]; spiral_fill(3) = [[1,2,3],[8,9,4],[7,6,5]].
C5	sell(stock, "rice", 4) gives ({"rice": 6, "beans": 5}, True); low_stock(stock, 6) = ["beans"]; total_items = 15.
C6	X wins, O wins (diagonal), Draw, In progress for the four examples in order.
E1	count_ways_dp(100, [1,5,10,25,50]) = 292; min_coins(11, [1,2,5]) = 3; min_coins(100, [1,5,10,25,50]) = 2.
E2	Sequence 1, 2, 1, 3, 2, 1 with max_size 2: LRU 1 hit and 5 misses; FIFO 2 hits and 4 misses.
E3	reachable(graph, "A") = 6; reachable(graph, "C") = 4; shortest paths from A to F have 3 edges.
E4	edit_distance("kitten", "sitting") = 3; ("flaw", "lawn") = 2; ("", "abc") = 3.

End of Week 9 Challenge